

DiabetesCare

DOCUMENTAÇÃO TÉCNICA DOS MÓDULOS WEB, MOBILE, BACKEND E ANALÍTICO

Projeto: DiabetesCare

Data: 21/04/2026

Grupo: Gabriel Pessoni, Hudson Ribeiro Barbara Junior e Livia Portela Ferreira

Franca SP

2026

RESUMO

O presente documento apresenta a documentação técnica do projeto DiabetesCare, desenvolvido no contexto do Projeto Interdisciplinar do quinto semestre do curso de Desenvolvimento de Software Multiplataforma. A solução proposta tem como foco o acompanhamento de informações relacionadas ao diabetes por meio de um aplicativo mobile, de um painel administrativo web, de uma API REST centralizada e de um módulo analítico em Python voltado à classificação de dados clínicos.

O objetivo principal deste documento consiste em registrar, de forma estruturada, a organização do repositório, as tecnologias empregadas, as funcionalidades implementadas e o estado atual de integração entre os módulos do sistema. A elaboração do material baseou-se na leitura técnica do repositório mais recente, incluindo os diretórios do aplicativo mobile, frontend web, backend e módulo analítico, bem como seus arquivos de configuração, rotas, controladores, contexto global, tema, modelagem de banco de dados e documentação principal do projeto.

A partir dessa análise, verificou-se que o projeto apresenta um ecossistema mais completo e integrado. O aplicativo mobile, desenvolvido com Expo e React Native, concentra a experiência do paciente. O painel web, construído com Next.js 14, oferece recursos administrativos e analíticos para acompanhamento clínico. O backend, implementado com Express, Prisma e PostgreSQL, centraliza autenticação, regras de negócio, persistência e documentação de API. Além disso, o módulo analítico em Python executa pré-processamento, classificação supervisionada e testes automatizados sobre o Pima Indians Diabetes Dataset.

Conclui-se que o repositório atual evidencia uma evolução significativa da solução, deixando de ser apenas um conjunto de protótipos desacoplados para assumir a forma de uma arquitetura em módulos integráveis, com base concreta para evolução funcional, persistência real de dados e ampliação dos recursos clínicos e analíticos.

SUMÁRIO

1 INTRODUÇÃO

1.1 Objetivo do Documento

1.2 Público-Alvo

2 VISÃO GERAL DO PROJETO

2.1 Visão Geral

2.2 Módulo Mobile

2.3 Módulo Web

2.4 Módulo Backend

2.5 Módulo Analítico

2.6 Integração entre os Módulos

3 TECNOLOGIAS UTILIZADAS

3.1 Tecnologias do Módulo Web

3.2 Tecnologias do Módulo Mobile

3.3 Tecnologias do Backend

3.4 Tecnologias do Módulo Analítico

3.5 Banco de Dados

3.6 Ferramentas Auxiliares

4 ARQUITETURA DO SISTEMA

4.1 Arquitetura Geral

4.2 Diagrama Descritivo

4.3 Comunicação entre Camadas

4.4 Padrões Adotados

5 ESTRUTURA DOS PROJETOS

5.1 Estrutura do Projeto Web

5.2 Estrutura do Projeto Mobile

5.3 Estrutura do Backend

5.4 Estrutura do Módulo Analítico

5.5 Organização de Código e Boas Práticas

6 FUNCIONALIDADES PRINCIPAIS

6.1 Funcionalidades do Sistema Web

6.2 Funcionalidades do Sistema Mobile

6.3 Funcionalidades do Backend

6.4 Funcionalidades do Módulo Analítico

7 REQUISITOS DO SISTEMA

7.1 Requisitos Funcionais

7.2 Requisitos Não Funcionais

8 SEGURANÇA

8.1 Autenticação e Autorização

8.2 Proteção de Dados

8.3 Boas Práticas de Segurança Adotadas

9 INTEGRAÇÕES

9.1 Integrações Atuais

9.2 Integrações Futuras

10 DEPLOY E INFRAESTRUTURA

10.1 Ambientes de Execução

10.2 Processo de Deploy

11 CONSIDERAÇÕES FINAIS

11.1 Pontos Fortes do Sistema

REFERÊNCIAS INTERNAS DO PROJETO

1 INTRODUÇÃO

O projeto DiabetesCare foi organizado como uma solução multiplataforma voltada ao acompanhamento de informações relacionadas ao cuidado com diabetes. O repositório analisado apresenta quatro frentes principais de desenvolvimento: um aplicativo mobile para uso do paciente, um painel web administrativo para visualização e gerenciamento de dados, uma API backend centralizada para autenticação e persistência e um módulo analítico em Python direcionado à classificação de pacientes com base em atributos clínicos.

A presente versão da documentação procura refletir o estado real do código atualmente disponível no repositório do projeto. Dessa forma, as descrições apresentadas a seguir foram ajustadas para representar com fidelidade as funcionalidades já implementadas, as integrações existentes e os pontos que ainda permanecem em evolução.

1.1 Objetivo do Documento

Este documento tem como finalidade apresentar, de forma técnica, estruturada e abrangente, a arquitetura, as tecnologias empregadas, os módulos, as funcionalidades e as diretrizes de desenvolvimento e operação da plataforma DiabetesCare.

Seu papel é servir como referência central para toda a equipe do projeto, assegurando que desenvolvedores, arquitetos de software, analistas de qualidade, equipes de infraestrutura e gestores tenham acesso a uma visão unificada, precisa e continuamente atualizável da solução.

Além disso, o documento também atua como material de integração técnica para novos membros da equipe, facilitando o processo de onboarding e acelerando a compreensão do domínio de negócio e das decisões arquiteturais adotadas ao longo da evolução do projeto.

1.2 Público-Alvo

Este documento é direcionado aos seguintes perfis:

- Desenvolvedores Frontend: estrutura de componentes, rotas, estilos e padrões de código.
- Desenvolvedores Mobile: navegação, estado global, autenticação, tema e integração com APIs.
- Desenvolvedores Backend: rotas, validações, segurança, persistência e documentação da API.
- Arquitetos de Software: decisões arquiteturais, padrões e escalabilidade.
- Engenheiros de Infraestrutura: deploy, ambientes, monitoramento e segurança.
- Gestores de Produto: funcionalidades, roadmap técnico e integrações.
- Analistas de Qualidade (QA): requisitos funcionais, fluxos críticos e testes.

2 VISÃO GERAL DO PROJETO

2.1 Visão Geral

Conforme indicado na documentação principal do projeto, o DiabetesCare foi concebido como um ecossistema de saúde digital voltado ao acompanhamento e gerenciamento do diabetes. A plataforma conecta pacientes por meio do aplicativo mobile, profissionais de saúde por meio do painel web, uma API REST centralizada para autenticação e persistência e um módulo de pré-diagnóstico baseado em machine learning.

A estrutura atual do projeto encontra-se dividida em quatro módulos principais: aplicativo mobile, painel web administrativo, backend API e módulo analítico em Python.

2.2 Módulo Mobile

O aplicativo mobile do DiabetesCare foi desenvolvido com Expo e React Native, sendo voltado ao paciente e ao acompanhamento diário do diabetes. O módulo reúne recursos voltados ao autocuidado, ao registro clínico e à visualização de informações pessoais de saúde.

A experiência mobile foi estruturada em torno de telas temáticas acessíveis por navegação principal e por fluxos complementares, contemplando acompanhamento glicêmico, alimentação, hidratação, medicações, metas, notificações, perfil e pré-diagnóstico.

Entre os principais módulos funcionais presentes no aplicativo, destacam-se:

- Início (Home): visão geral do dia, incluindo glicose, medicações, ações rápidas e progresso da água.
- Glicose: registro e histórico de medições com status colorido e gráfico de evolução.
- Alimentação: diário alimentar por tipo de refeição, com macronutrientes e metas calóricas.
- Diário: registro de humor, sintomas, tags e reflexões diárias.
- Medicamentos: lista de medicações com horários, aderência e registro de dose tomada.
- Metas: configuração e acompanhamento de metas de saúde personalizadas.
- Hidratação: controle de ingestão diária de água com meta configurável.
- Relatórios: visualização de histórico e evolução dos indicadores de saúde.
- Notificações: central de alertas, dicas e lembretes do sistema.
- Configurações: preferências do aplicativo, perfil, unidades e suporte.
- FAQ: banco de perguntas frequentes sobre diabetes e uso do aplicativo.
- Dicas e Artigos: conteúdo educativo categorizado.
- Diagnóstico / Pré-diagnóstico: fluxo de apoio clínico e análise inicial.

2.3 Módulo Web

Painel administrativo construído com Next.js 14, voltado ao acompanhamento clínico, visualização de indicadores, dashboards e gestão centralizada. O frontend web oferece visualização de pacientes, glicose, alimentação e indicadores de saúde, mantendo layout administrativo reutilizável com sidebar e header dinâmico.

2.4 Módulo Backend

A API REST do projeto foi construída com Express, Prisma e PostgreSQL, sendo responsável por autenticação, persistência de dados, validação de entradas, regras de negócio e documentação técnica da aplicação. Essa camada centraliza o acesso ao banco de dados e fornece a base para integração entre os demais módulos do ecossistema DiabetesCare.

Além do papel de intermediar a comunicação entre cliente e servidor, o backend organiza os principais fluxos funcionais do sistema, como autenticação de usuários, gerenciamento de dados clínicos, operações administrativas e controle de acesso às rotas protegidas. Sua estrutura em camadas contribui para a separação de responsabilidades, facilitando a manutenção, os testes automatizados e a evolução do projeto.

No estado atual do sistema, o backend já representa uma base concreta de integração, persistência e segurança, permitindo que os demais módulos evoluam progressivamente de uma abordagem apoiada em mock data para uma arquitetura mais consolidada e orientada a dados reais.

2.5 Módulo Analítico

O módulo analítico foi desenvolvido em Python e é voltado ao pré-processamento de dados, à classificação supervisionada e à inferência de risco de diabetes, operando como recurso complementar de apoio analítico dentro do ecossistema DiabetesCare.

Esse componente concentra scripts responsáveis pelo tratamento do conjunto de dados, limpeza de valores inconsistentes, normalização, comparação entre classificadores e geração de inferências com base em atributos clínicos. Sua finalidade é apoiar estudos, experimentos e análises relacionadas ao pré-diagnóstico, ampliando o escopo técnico do projeto para além da interface e da persistência de dados.

No estado atual, o módulo analítico permanece desacoplado da aplicação principal, funcionando de forma independente em relação aos módulos mobile, web e backend. Ainda assim, sua presença representa uma base importante para futuras integrações com recursos preditivos e apoio clínico baseado em dados.

2.6 Integração entre os Módulos

A integração geral do ecossistema DiabetesCare parte de uma arquitetura em que o aplicativo mobile e o painel web consomem dados da API backend, enquanto o backend realiza autenticação, validação de entradas, aplicação das regras de negócio e persistência das informações no banco PostgreSQL.

Nesse modelo, o backend atua como camada central de comunicação entre os módulos principais, fornecendo a base para integração progressiva entre as interfaces e os dados persistidos. Essa organização contribui para a separação de responsabilidades, melhora a manutenção da solução e favorece a evolução do projeto em direção a uma arquitetura mais consolidada e orientada a serviços.

O módulo analítico, por sua vez, permanece desacoplado da camada principal da aplicação, operando como componente standalone de apoio ao pré-diagnóstico e à análise de risco. Embora ainda não esteja integrado diretamente ao fluxo operacional dos demais módulos, sua existência amplia o escopo técnico da plataforma e estabelece base para futuras integrações analíticas.

2.7 Integração entre os Sistemas

Os módulos principais do DiabetesCare foram concebidos para operar de forma integrada dentro de um mesmo ecossistema. O aplicativo mobile atua como principal interface de entrada e acompanhamento das informações do paciente, enquanto o painel web desempenha papel administrativo e analítico na visualização e no gerenciamento desses dados.

A integração entre os sistemas ocorre por meio de uma API REST centralizada, responsável por intermediar a comunicação entre as interfaces e a camada de persistência. De forma geral, esse fluxo ocorre da seguinte maneira:

1. O aplicativo mobile envia dados clínicos e informações do usuário por meio de requisições HTTP autenticadas.
2. A API realiza validação de entrada, autenticação, aplicação das regras de negócio e persistência dos dados no banco PostgreSQL.
3. O painel web consome dados da API para exibição de indicadores, tabelas, dashboards e informações administrativas.

4. O backend fornece a base necessária para evolução progressiva da integração entre os módulos, permitindo substituir gradualmente fluxos ainda apoiados em mock data por consumo real de dados persistidos.

Embora a arquitetura já esteja definida para integração entre mobile, web e backend, parte das interfaces ainda se encontra em processo de transição entre prototipação visual e consumo pleno da API. Ainda assim, o ecossistema atual já estabelece uma base consistente para comunicação entre os módulos principais da plataforma.

3 TECNOLOGIAS UTILIZADAS

3.1 Frontend Web

O painel administrativo web foi desenvolvido com Next.js 14 e organizado com App Router, utilizando uma stack moderna baseada em React, TypeScript e Tailwind CSS. O módulo foi estruturado para oferecer visualização clínica, dashboards, tabelas administrativas e navegação entre módulos do sistema, com foco em componentização, consistência visual e experiência de uso.

Tecnologia	Versão	Finalidade
Next.js	14.2.5	Framework React com SSR, SSG e App Router
React	18.x	Base para construção de interfaces
TypeScript	5.x	Tipagem estática e segurança
Tailwind CSS	3.4.1	Estilização utilitária e design system
Recharts	2.12.7	Gráficos responsivos baseados em D3.js
lucide-react	0.400.0	Ícones SVG consistentes e leves
clsx	2.1.1	Composição condicional de classes CSS
tailwind-merge	2.5.2	Resolução de conflitos Tailwind

Design System Web

Token	Valor	Uso
Primary Green	#4CAF82	Ações primárias e indicadores positivos
Primary Dark	#388E63	Hover de elementos primários
Primary Light	#E8F5EE	Fundos de badges e cards destacados
Secondary Blue	#3B8ED0	Informações e links secundários
Warning Orange	#F59E0B	Alertas e valores intermediários
Danger Red	#EF4444	Erros, valores críticos e ações destrutivas
Success Green	#10B981	Confirmações e status positivos
Background	#F7F9FC	Fundo geral da aplicação
Text Primary	#1A2332	Texto principal e títulos
Text Secondary	#6B7280	Descrições e rótulos secundários
Border	#E5E7EB	Bordas de cards e inputs

3.2 Backend e Dados

O backend do projeto foi desenvolvido em Node.js com TypeScript, utilizando arquitetura modular para organização das rotas, controladores, middlewares e acesso a dados. A API centraliza a lógica principal da plataforma, atuando como camada de integração entre o painel web, aplicativo mobile e banco de dados PostgreSQL.

A persistência relacional é realizada por meio do Prisma ORM, responsável pela modelagem das entidades, migrations e acesso tipado ao banco. O backend também conta com autenticação baseada em JWT, documentação Swagger para inspeção dos endpoints e utilização de bibliotecas auxiliares de segurança e validação.

Tecnologia	Finalidade
Node.js	Runtime JavaScript server-side
Fastify / Express	Framework HTTP para API RESTful
TypeScript	Tipagem no backend e consistência com o frontend
Prisma ORM	ORM e migrations de banco de dados
JWT (JSON Web Tokens)	Autenticação stateless entre cliente e servidor
Zod	Validação de schemas na API
bcrypt	Hash seguro de senhas
Helmet	Proteção de headers HTTP
CORS	Controle de acesso entre origens
Swagger UI	Documentação da API

3.3 Mobile

O aplicativo mobile foi desenvolvido com Expo e React Native, utilizando TypeScript, React Navigation, AsyncStorage, bibliotecas de interface e componentes gráficos auxiliares. Embora Zustand esteja instalado como dependência, o gerenciamento principal de estado observado no projeto está concentrado em AppContext, construído com React Context API. O projeto também possui AuthContext para o controle do estado de autenticação.

Tecnologia	Versão	Finalidade
Expo	~54.0.33	Plataforma gerenciada para React Native
React Native	0.81.5	Framework para apps nativos multiplataforma
React	19.1.0	Base para construção de interfaces
TypeScript	~5.9.2	Tipagem estática
@react-navigation/native	^7.2.2	Navegação principal do aplicativo
@react-navigation/bottom-tabs	^7.15.9	Navegação por abas inferiores
@react-navigation/native-stack	^7.14.10	Navegação em pilha
Zustand	^5.0.12	Estado auxiliar; não é o mecanismo principal atual
expo-linear-gradient	~15.0.8	Gradientes lineares nativos
react-native-safe-area-context	~5.6.0	Tratamento de áreas seguras
lucide-react-native	^1.7.0	Ícones SVG para React Native
expo-status-bar	~3.0.8	Controle da barra de status
AsyncStorage	2.2.0	Persistência local de dados
react-native-svg	15.12.1	Base para componentes gráficos

Design System Mobile

Token	Tamanho	Uso típico
xs	11px	Labels de tags e badges
sm	13px	Textos auxiliares e notas
md	15px	Corpo de texto principal
lg	17px	Subtítulos de seção
xl	20px	Títulos de card
xxl	24px	Títulos de tela
xxxl	28px	Headers de destaque
huge	34px	Números grandes em KPIs
Token	Valor	Uso típico
xs	4px	Espaçamentos mínimos
sm	8px	Padding de badges/separadores
md	12px	Padding de cards compactos
lg	16px	Padding padrão de cards
xl	20px	Margens entre seções
xxl	24px	Espaçamentos de tela
huge	48px	Margens e espaços grandes

3.4 Banco de Dados

A camada de dados persistente do projeto é implementada com PostgreSQL, acessado por meio do Prisma ORM. A arquitetura do ecossistema já define o backend como responsável pela persistência relacional, utilizando Prisma como camada de modelagem e acesso ao banco.

A estrutura do backend contempla schema de dados, migrations e configuração voltada à evolução controlada da base. Essa modelagem relacional fornece a base para autenticação, persistência e expansão funcional do ecossistema, sustentando os dados clínicos e administrativos da plataforma.

Componente	Tecnologia	Justificativa
Banco de dados relacional	PostgreSQL	Persistência principal da aplicação
ORM	Prisma ORM	Modelagem, migrations e acesso tipado
Schema	Prisma Schema	Definição estruturada das entidades
Seed	Prisma Seed	Popular dados iniciais de desenvolvimento

Entidade	Descrição
Usuario	Armazena cadastro, credenciais, perfil de acesso e dados clínicos básicos do usuário
Diario	Registro textual e emocional com título, conteúdo, humor, sintomas e tags
Dicas	Conteúdo educativo com título, sumário, categoria, tempo de leitura e destaque
FAQ	Perguntas frequentes com categoria, ordem e status de ativação
Sono	Registro de sono com horários, duração, qualidade e notas
Glicose	Registro glicêmico com valor, contexto, data, hora, status e observações
Meta	Meta de saúde com categoria, alvo, valor atual, unidade, prazo, cor e status
Hidratacao	Ingestão hídrica com data, hora e quantidade consumida

3.5 APIs e Integrações

Integração	Finalidade
API REST interna	Comunicação entre mobile, web e backend
PostgreSQL + Prisma ORM	Persistência e modelagem relacional
Swagger UI	Documentação interativa da API
JWT	Autenticação stateless cliente-servidor

3.6 Ferramentas Auxiliares

Ferramenta	Finalidade
Git + GitHub	Controle de versão e colaboração
ESLint	Padronização e análise estática
Prettier	Formatação automática de código
Swagger UI	Documentação interativa da API
Jest	Testes automatizados
Supertest	Testes de integração da API
Prisma CLI	Migrations, seed e geração do client
Expo	Execução e testes do app mobile
Python Scripts	Execução do módulo analítico
GitHub Actions	Automação parcial de fluxos técnicos

4 ARQUITETURA DO SISTEMA

4.1 Arquitetura Geral

A plataforma DiabetesCare adota uma arquitetura modular em camadas, com separação clara entre frontend mobile, frontend web, backend REST e módulo analítico em Python.

A camada de apresentação é composta pelo aplicativo mobile em Expo/React Native e pelo painel administrativo em Next.js. A camada de aplicação é representada pela API REST construída com Express, responsável por autenticação, validação, regras de negócio e documentação Swagger. A camada de dados é implementada com PostgreSQL, acessado por meio do Prisma ORM. De forma complementar, o módulo analítico em Python opera como componente standalone para apoio ao pré-diagnóstico e à classificação de risco.

No estado atual do projeto, o backend já se encontra implementado e funcional como base de integração. Ainda assim, os frontends mantêm partes do fluxo apoiadas em mock data, caracterizando uma fase intermediária de acoplamento entre as camadas do ecossistema.

4.2 Diagrama Descritivo

O ecossistema DiabetesCare é composto por quatro módulos principais. O aplicativo mobile, desenvolvido com Expo e React Native, e o painel web administrativo, desenvolvido com Next.js 14, comunicam-se com o backend por meio de requisições HTTP/REST autenticadas com JWT. O backend, implementado com Express e Prisma ORM, centraliza autenticação, validação, regras de negócio e persistência de dados em PostgreSQL. De forma complementar, o módulo analítico em Python opera de maneira independente, como componente standalone voltado ao pré-diagnóstico e à análise de risco.

4.3 Comunicação entre Camadas

A comunicação entre os módulos principais do DiabetesCare ocorre por meio de requisições HTTP/REST autenticadas com JSON Web Token (JWT). O aplicativo mobile e o painel web utilizam a API backend como camada central de acesso aos dados, autenticação e regras de negócio.

No aplicativo mobile, a integração com a API ocorre de forma progressiva, coexistindo com fluxos ainda apoiados em dados em memória e Context API. No painel web, parte significativa das páginas ainda utiliza dados mockados locais, embora a arquitetura já esteja preparada para consumo progressivo da API central.

O backend atua como camada intermediária entre as interfaces e o banco PostgreSQL, sendo responsável por autenticação, validação de entradas, persistência de dados e disponibilização de endpoints documentados via Swagger. Dessa forma, a comunicação entre as camadas já se encontra estruturalmente definida, ainda que parte dos módulos de interface permaneça em transição entre prototipação e integração plena com os dados persistidos.

4.4 Padrões Adotados

Sistema Web (Next.js)

Padrão	Aplicação
File-based Routing	Rotas definidas por pastas no App Router
Route Groups	Agrupamento (admin) para layout compartilhado
Component Composition	Componentes reutilizáveis, como Card, Table e Badge
Single Responsibility	Cada componente possui responsabilidade única
Utility-First CSS	Tailwind CSS com classes via clsx/twMerge
Type-Safe Data Layer	Interfaces TypeScript para os dados

Sistema Mobile (React Native)

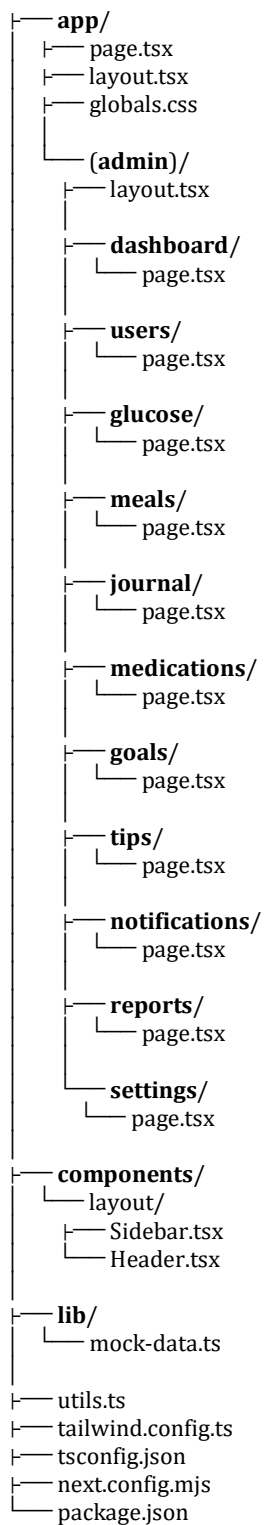
Padrão	Aplicação
Feature-Based Structure	Código organizado por funcionalidade
React Context API	Estado global no AppContext.tsx e AuthContext.tsx para autenticação
Custom Hooks	Lógica de negócio reutilizável
Theme Token System	Design system com tokens visuais
Navigator Pattern	Stack e Tab Navigator compostos hierarquicamente
StyleSheet.create	Estilos nativos para reaproveitamento e consistência

5 ESTRUTURA DOS PROJETOS

5.1 Estrutura do Projeto Web

A estrutura do módulo web:

web/



5.2 Estrutura do Projeto Mobile

A estrutura do módulo mobile:

app/

|— App.tsx

- index.ts
- app.json
- package.json
- tsconfig.json
- **assets/**
 - adaptive-icon.png
 - favicon.png
 - icon.png
 - splash-icon.png
- src/
 - **screens/**
 - Auth/
 - LoginScreen.tsx
 - RegisterScreen.tsx
 - Home/
 - HomeScreen.tsx
 - Glucose/
 - GlucoseScreen.tsx
 - AddGlucoseScreen.tsx
 - Food/
 - FoodDiaryScreen.tsx
 - AddFoodScreen.tsx
 - Journal/
 - JournalScreen.tsx
 - AddJournalScreen.tsx
 - Profile/
 - ProfileScreen.tsx
 - EditProfileScreen.tsx
 - More/
 - MoreScreen.tsx
 - Diagnosis/
 - DiagnosisScreen.tsx
 - DiagnosisDetailScreen.tsx
 - Tips/
 - TipsScreen.tsx
 - TipDetailScreen.tsx
 - Medications/
 - MedicationsScreen.tsx
 - Water/
 - WaterScreen.tsx
 - Goals/
 - GoalsScreen.tsx
 - Reports/
 - ReportsScreen.tsx
 - Notifications/
 - NotificationsScreen.tsx
 - Settings/
 - SettingsScreen.tsx
 - Onboarding/
 - OnboardingScreen.tsx
 - FAQ/
 - FAQScreen.tsx
 - Sleep/
 - SleepScreen.tsx
 - AddSleepScreen.tsx
 - **components/**
 - common/
 - Button.tsx
 - Card.tsx
 - EmptyState.tsx
 - GlucoseStatusBadge.tsx
 - ProgressBar.tsx
 - ScreenHeader.tsx
 - charts/
 - GlucoseChart.tsx
 - **navigation/**
 - AppNavigator.tsx
 - TabNavigator.tsx
 - context/
 - AppContext.tsx
 - AuthContext.tsx
 - **data/**

```

├── diagnosisData.ts
├── faqData.ts
├── foodDatabase.ts
├── mockData.ts
├── theme/
│   ├── colors.ts
│   └── index.ts
├── types/
└── utils/

```

5.3 Backend

backend/

```

├── src/
│   ├── controllers/
│   │   ├── authController.ts
│   │   ├── glucoseController.ts
│   │   └── userController.ts
│   ├── routes/
│   │   ├── authRoutes.ts
│   │   ├── glucoseRoutes.ts
│   │   └── userRoutes.ts
│   ├── middlewares/
│   │   ├── authMiddleware.ts
│   │   └── errorMiddleware.ts
│   ├── services/
│   │   ├── authService.ts
│   │   └── glucoseService.ts
│   ├── schemas/
│   │   ├── authSchema.ts
│   │   └── glucoseSchema.ts
│   └── app.ts
├── prisma/
│   ├── schema.prisma
│   ├── seed.ts
│   └── migrations/
├── tests/
├── package.json
└── tsconfig.json

```

5.4 Estrutura do Módulo Analítico:

algoritmo-python/

```

├── dataset/
├── scripts/
│   ├── preprocessamento.py
│   ├── treinamento.py
│   └── inferencia.py
├── models/
├── notebooks/
├── requirements.txt
├── main.py
└── README.md

```

5.5 Organização de Código e Boas Práticas

As seguintes boas práticas orientam a organização de código dos módulos principais do projeto.

Nomenclatura

- Componentes: PascalCase, como UserCard e GlucoseChart.
- Funções e variáveis: camelCase, como formatDate e glucoseStatusColor.
- Constantes: UPPER_SNAKE_CASE, como MOCK_USERS e GLUCOSE_TREND.
- Tipos e interfaces: PascalCase com prefixo semântico quando necessário.
- Arquivos de componentes: PascalCase com extensão .tsx.
- Arquivos utilitários: camelCase com extensão .ts.

Tipagem

- Uso de TypeScript nos módulos principais do projeto.
- Interfaces explícitas para os dados de domínio.
- Busca-se reduzir o uso de tipagens genéricas e privilegiar definições explícitas de tipos e interfaces.
- Uso de tipos utilitários do TypeScript, como Partial, Record e Pick, sempre que aplicável.

Componentes

- Utilização de componentes funcionais com React Hooks.
- Props tipadas com interfaces TypeScript.
- Separação clara entre lógica, concentrada em hooks e utilitários, e apresentação, concentrada nos componentes.
- Sempre que necessário, componentes maiores devem ser reorganizados em subcomponentes para favorecer legibilidade e manutenção.

Estado e Efeitos

- React Context API como mecanismo principal de estado global no mobile, centralizado em AppContext.tsx e complementado por AuthContext.tsx para autenticação.
- useState e useReducer para estados locais de interface.
- useEffect utilizado com cautela e com declaração correta do array de dependências.

A organização modular adotada no projeto contribui para maior separação de responsabilidades entre os componentes do sistema, favorecendo manutenção, escalabilidade e evolução progressiva da plataforma.

6 FUNCIONALIDADES PRINCIPAIS

6.1 Funcionalidades do Sistema Web

6.1.1 Dashboard Principal

- KPIs de topo: total de usuários cadastrados, glicose média das últimas 24 horas, progresso de metas ativas e quantidade de artigos publicados.
- Gráfico de tendência glicêmica: area chart com valores mínimo, médio e máximo dos últimos sete dias para o usuário monitorado.
- Alertas recentes: painel lateral com os últimos eventos críticos, categorizados por severidade: Perigo, Atenção, Info e Sucesso.
- Distribuição glicêmica: gráfico de pizza com proporção de leituras em cada faixa de status.
- Distribuição por tipo de diabetes: visualização proporcional dos tipos de diabetes presentes na base de usuários.
- Consumo calórico semanal: gráfico de barras com calorias diárias dos últimos sete dias.

- Tabela de últimas leituras: lista das medições mais recentes, com identificação do usuário e status colorido.
- Usuários ativos: lista rápida dos pacientes ativos, contendo HbA1c e glicose média.

6.1.2 Gestão de Usuários

- Visualização tabular de todos os pacientes cadastrados, com dados clínicos resumidos.
- Filtros por status, ativo ou inativo, e busca por nome ou e-mail.
- Indicadores visuais de HbA1c com código de cores verde, amarelo e vermelho.
- Barra de progresso de aderência medicamentosa por paciente.
- Cards de detalhamento com dados demográficos, médico responsável e último check-up.
- Botão para cadastro de novo usuário.

6.1.3 Monitoramento de Glicose

- Cards de KPI: média geral, percentual dentro da meta, percentual acima e percentual abaixo.
- Gráfico de evolução diária com linhas de referência clínica.
- Tabela de leituras com colunas para Usuário, Data/Hora, Valor, Contexto, Status e Notas.
- Filtros por status glicêmico e período temporal.
- Busca textual por usuário ou notas.

6.1.4 Diário Alimentar

- KPIs de calorias médias, carboidratos médios, refeições registradas e dias com registro.
- Gráfico de calorias por dia da semana.
- Breakdown de macronutrientes com barras de progresso.
- Lista de refeições agrupadas por tipo: café, almoço, jantar e lanche.
- Filtros por tipo de refeição e busca por alimento.

6.1.5 Diário Emocional

- Monitoramento do bem-estar psicológico dos pacientes.
- Gráfico de distribuição de humor: Ótimo, Bem, Ok, Mal e Péssimo.
- Gráfico de evolução do humor médio semanal em escala de 1 a 5.
- Lista de entradas com emoji de estado, conteúdo, sintomas e tags.
- Filtros por humor e busca por título ou conteúdo.

6.1.6 Controle de Medicamentos

- KPIs de total de medicamentos, tomados hoje, pendentes e aderência média.
- Distribuição por tipo, oral, insulina e suplemento, em gráfico de pizza.
- Histórico de aderência semanal em gráfico de barras.
- Tabela completa com nome, dosagem, frequência, horários programados e status de tomada.
- Cards de aderência por paciente com barra de progresso e código de cores.

6.1.7 Metas de Saúde

- Visão geral de todas as metas ativas, concluídas e atrasadas.
- Barras de progresso para cada meta com percentual calculado dinamicamente.
- Filtros por categoria, incluindo glicose, peso, exercício, água, sono e passos, e por status.
- Detecção automática de metas com prazo vencido, com sinalização visual.

- Tabela contendo meta, progresso, prazo, usuário e status.

6.1.8 Dicas & Artigos

- Gestão completa de conteúdo educativo da plataforma.
- Cards com categoria colorida, métricas de visualizações e curtidas e badge de destaque.
- Filtros por categoria e toggle de conteúdo em destaque.
- Gráfico de engajamento, comparando visualizações e curtidas por artigo.
- Ações de editar e visualizar diretamente nos cards.
- Botão de criação de novo artigo.

6.1.9 Central de Notificações

- Histórico completo de notificações enviadas com status de leitura.
- Filtros por tipo - glicose, refeição, medicação, consulta, dica e meta - e por status, lida ou não lida.
- Gráfico de notificações enviadas versus lidas por dia da semana.
- Identificação visual de mensagens não lidas com destaque na linha.
- Botão para reenvio e criação de nova notificação manual.

6.1.10 Relatórios e Analytics

- Painel consolidado com KPIs comparativos, incluindo variação em relação aos seis meses anteriores.
- Gráfico de crescimento de usuários mês a mês.
- Evolução da glicose média mensal versus meta clínica.
- Histórico de aderência medicamentosa mensal.
- Distribuição de HbA1c por faixa de controle.
- Ranking de pacientes por score de controle clínico calculado.
- Grid de exportação de relatórios por área, com suporte a CSV e PDF.

6.1.11 Configurações do Sistema

- Perfil: dados do administrador, foto e papel, como Administrador, Moderador ou Visualizador.
- Notificações: toggles individuais por tipo de alerta e configuração de limites clínicos.
- Segurança: troca de senha, autenticação em dois fatores e gerenciamento de sessões ativas.
- Sistema: parâmetros clínicos padrão, idioma, fuso horário e status dos serviços.
- Aparência: tema claro ou escuro, cor de destaque e preferências de layout.

6.2 Funcionalidades do Sistema Mobile

6.2.1 Onboarding

- Fluxo de apresentação para novos usuários com configuração do perfil clínico.
- Coleta de dados como tipo de diabetes, glicose alvo, médico responsável, peso e altura.
- Persistência do estado de onboarding com AsyncStorage.

6.2.2 Tela Inicial (Home)

- Saudação personalizada conforme o período do dia: bom dia, boa tarde ou boa noite.
- Card de status atual da glicose com cor indicativa.
- Botões de ação rápida: Registrar Glicose, Registrar Refeição, Registrar Água e Medicação.

- Resumo do dia, contemplando refeições, leituras e hidratação.
- Gráfico compacto de glicose das últimas medições.
- Tracker de água com progresso visual da meta diária.
- Contador de medicações do dia.

6.2.3 Monitoramento de Glicose

- Registro de leituras com valor, contexto - jejum, pré-refeição, pós-refeição, antes de dormir ou aleatório - e notas.
- Cards de estatísticas: média, percentual dentro da meta, percentual acima e percentual abaixo.
- Gráfico de evolução temporal.
- Lista de histórico com badges coloridas de status.
- Filtros por período: Hoje, Semana e Mês.
- Swipe para exclusão de registros.

6.2.4 Diário Alimentar

- Registro de refeições por tipo, com seleção de alimentos e quantidades.
- Cálculo automático de macronutrientes e calorias.
- Totais diários com barras de progresso em relação à meta.
- Agrupamento de refeições por tipo com ícones e cores distintas.
- Seleção de data: Hoje, Ontem e Anteontem.
- Swipe para exclusão de entradas.

6.2.5 Diário Emocional

- Registro de entradas com título, conteúdo livre, seleção de humor e sintomas.
- Listagem com emoji de humor, preview do conteúdo e data.
- Filtragem por humor.

6.2.6 Controle de Medicamentos

- Lista de medicamentos com horários programados e status, tomado ou pendente.
- Ação de marcar medicação como tomada.
- Diferenciação visual por tipo: oral em verde, insulina em roxo e suplemento em laranja.
- Registro do horário de tomada.

6.2.7 Controle de Hidratação

- Registro de ingestão de água por porção.
- Barra de progresso circular rumo à meta diária.
- Histórico de consumo do dia.

6.2.8 Metas de Saúde

- Listagem de metas por categoria com barra de progresso.
- Atualização de progresso diretamente pelo aplicativo.
- Metas concluídas com destaque visual de conquista.

6.2.9 Perfil do Paciente

- Visualização e edição de dados clínicos pessoais.
- Histórico de HbA1c e informações do médico responsável.

- Configuração de metas de glicose personalizadas.

6.2.10 Relatórios

- Gráficos de evolução histórica dos principais indicadores.
- Resumo semanal e mensal de glicose, alimentação e aderência.

6.2.11 Notificações

- Lista de alertas, lembretes e dicas com status de leitura.
- Marcação individual ou em massa como lida.
- Badge com contagem de não lidas na aba de navegação.

6.2.12 Dicas & Artigos

- Listagem categorizada de conteúdo educativo.
- Visualização de artigo com tempo estimado de leitura.
- Categorias: Exercício, Alimentação, Emergência e Bem-estar.

6.2.13 FAQ

- Banco de perguntas frequentes sobre diabetes e uso do aplicativo.
- Interface em formato accordion, permitindo expandir e recolher respostas.

6.2.14 Configurações

- Preferências de notificação por tipo.
- Idioma e unidades de medida.
- Tema do aplicativo.
- Informações de suporte e sobre o aplicativo.

7 REQUISITOS DO SISTEMA

7.1 Requisitos Funcionais

Sistema Web

ID	Requisito	Prioridade
RFW01	Deve exibir um dashboard com KPIs em tempo real	Alta
RFW02	Admin deve visualizar e filtrar pacientes	Alta
RFW03	Deve exibir leituras glicêmicas com cores por status	Alta
RFW04	Admin deve criar, editar e excluir artigos educativos	Alta
RFW05	Deve permitir envio manual de notificações	Média
RFW06	Deve gerar relatórios em CSV e PDF	Média
RFW07	Admin deve configurar limites de alerta clínico	Média
RFW08	Deve exibir alertas para valores glicêmicos críticos	Alta
RFW09	Deve suportar perfis de acesso, como Admin e Moderador	Média
RFW10	Deve exibir histórico de aderência por paciente	Alta

Sistema Mobile

ID	Requisito	Prioridade
RFM01	Paciente deve registrar glicose com contexto	Alta
RFM02	App deve calcular macronutrientes das refeições	Alta
RFM03	App deve enviar lembretes de medicação nos horários	Alta
RFM04	Paciente deve registrar humor e sintomas diariamente	Média
RFM05	App deve exibir gráficos de evolução glicêmica	Alta
RFM06	App deve suportar metas de saúde personalizáveis	Média
RFM07	App deve funcionar offline com conectividade intermitente	Alta
RFM08	O paciente deve concluir onboarding na primeira abertura	Alta
RFM09	App deve exibir artigos educativos categorizados	Média
RFM10	App deve integrar HealthKit (iOS) e Google Fit (Android)	Baixa

7.2 Requisitos Não Funcionais

Performance

ID	Requisito
RNFP01	Painel web deve carregar tela inicial em menos de 2s, com LCP inferior a 2s
RNFP02	API deve responder em menos de 300ms em 95% das requisições
RNFP03	App mobile deve renderizar telas em menos de 100ms
RNFP04	Listas grandes devem utilizar FlatList virtualizada para evitar jank
RNFP05	Imagens devem ser comprimidas e entregues via CDN

Segurança

ID	Requisito
RNFS01	Comunicações via HTTPS com TLS 1.3
RNFS02	Senhas com bcrypt, fator 12 ou superior
RNFS03	JWT expira em 15 minutos, com refresh token de 7 dias
RNFS04	App mobile armazena tokens com expo-secure-store
RNFS05	API deve implementar rate limiting por IP e usuário autenticado
RNFS06	Dados de saúde criptografados em repouso

Escalabilidade

ID	Requisito
RNFE01	Arquitetura suporta crescimento horizontal da API
RNFE02	Banco suporta até 100.000 usuários sem degradação
RNFE03	Notificações processam até 10.000 pushes simultâneos
RNFE04	Cache Redis para queries frequentes e sessões

Usabilidade

ID	Requisito
RNFU01	Painel web deve ser responsivo a partir de 1024px
RNFU02	O app mobile deve suportar iOS 15+ e Android 10+
RNFU03	App deve seguir WCAG 2.1 Nível AA
RNFU04	Ações destrutivas solicitam confirmação do usuário

Disponibilidade

ID	Requisito
RNFD01	Deve ter disponibilidade mínima de 99,5% conforme SLA
RNFD02	Manutenções agendadas fora do horário de pico
RNFD03	Deve ter backup diário com retenção de 30 dias

8 SEGURANÇA

8.1 Autenticação e Autorização

Fluxo de Autenticação

O sistema utiliza autenticação baseada em JWT (JSON Web Tokens) com estratégia de refresh token.

1. Login: o usuário envia credenciais, e-mail e senha, para o endpoint /auth/login.
2. Validação: a API verifica as credenciais comparando com o hash bcrypt armazenado no banco.
3. Emissão de tokens: a API emite dois tokens: Access Token, válido por 15 minutos, utilizado em todas as requisições autenticadas; e Refresh Token, válido por 7 dias, armazenado em cookie HttpOnly no web ou secure store no mobile.
4. Renovação: quando o Access Token expira, o cliente realiza requisição silenciosa ao endpoint /auth/refresh para obter um novo par de tokens.
5. Logout: o Refresh Token é invalidado no banco de dados por blacklist baseada em JTI.

Controle de Acesso (RBAC)

Papel	Permissões
Administrador	Acesso total: leitura, escrita, exclusão e configuração
Moderador	Leitura geral e escrita em conteúdos, artigos e notificações
Visualizador	Somente leitura, sem alteração de dados

Autenticação Mobile

- Tokens armazenados com expo-secure-store, utilizando encrypted keychain no iOS e keystore no Android.
- Renovação automática, transparente ao usuário.
- Logout automático após a expiração do Refresh Token.
- Biometria opcional, como Face ID ou Touch ID, para abertura do app.

8.2 Proteção de Dados

O sistema trata dados de saúde como dados sensíveis, seguindo as diretrizes da LGPD (Lei Geral de Proteção de Dados - Lei 13.709/2018) e as melhores práticas de segurança da informação.

Dados em Trânsito

- Toda comunicação ocorre via HTTPS com TLS 1.3.
- Certificados SSL/TLS gerenciados automaticamente via Let's Encrypt ou provedor cloud.
- Headers obrigatórios de segurança HTTP, incluindo HSTS, X-Frame-Options e Content-Security-Policy.

Dados em Repouso

- Senhas armazenadas exclusivamente como hash bcrypt, com custo mínimo 12.

- Campos sensíveis de saúde, como diagnóstico e histórico, criptografados com AES-256 no banco.
- Backups do banco de dados criptografados antes do armazenamento.

Consentimento e Privacidade

- Fluxo de consentimento explícito na primeira abertura do aplicativo.
- Política de privacidade acessível a qualquer momento nas configurações.
- Funcionalidade de exportação e exclusão de dados pelo próprio paciente, em conformidade com o direito ao esquecimento.

8.3 Boas Práticas de Segurança Adotadas

Prática	Implementação
Validação de entrada	Inputs validados com Zod no backend e frontend
Sanitização de dados	Prevenção de XSS com escape HTML e CSP
Rate Limiting	Limite de requisições por IP, ex.: 100 req/min no login
Proteção CSRF	Tokens CSRF e cookies SameSite
SQL Injection	Prisma ORM com queries parametrizadas
Logs de auditoria	Registro de ações administrativas com timestamp e IP
Gerenciamento de dependências	Atualizações com npm audit e Dependabot
Segredos de ambiente	Variáveis sensíveis em .env com acesso restrito
2FA opcional	2FA via TOTP, como Google Auth

9 INTEGRAÇÕES

9.1 APIs Externas

Firestore Cloud Messaging (FCM)

- Trigger: a API backend detecta eventos de gatilho, como glicose crítica, horário de medicação ou metas.
- Fluxo: Evento -> Job no Redis -> Worker -> FCM API -> Dispositivo do paciente.
- Payload: { title, body, data: { type, userId, entityId } }, com suporte a deep-link para a tela relevante.
- Certificados: APNs para iOS e credenciais de serviço Google para Android.

Apple HealthKit (iOS)

- Dados lidos: passos, frequência cardíaca, peso, sono e calorias.
- Dados escritos: glicose registrada no app sincronizada com o HealthKit.
- Permissões: solicitadas individualmente ao usuário, com descrição clara da finalidade de uso.

Google Fit / Health Connect (Android)

- Dados lidos: passos, frequência cardíaca e calorias ativas.
- Autenticação: OAuth2 com escopo restrito aos dados necessários.

SendGrid / Resend (E-mail)

- Envio automático de relatórios semanais em PDF ao administrador.
- Envio de alertas críticos por e-mail quando pacientes atingem valores glicêmicos extremos.
- Notificações de inatividade prolongada.

9.2 Serviços de Terceiros

Serviço	Finalidade
Vercel	Deploy e hosting web com CDN global
Expo EAS	Build, assinatura e distribuição do app mobile
AWS S3 / R2	Armazenamento de imagens e exportações
Sentry	Monitoramento de erros web e mobile
Datadog	Métricas, APM e dashboards de operação
Let's Encrypt	Certificados SSL/TLS automatizados

9.3 Comunicação entre Web e Mobile

Embora web e mobile sejam módulos distintos, ambos compartilham a mesma API backend e o mesmo ecossistema de dados. A comunicação entre as interfaces e o backend ocorre por meio de requisições HTTP/REST autenticadas com JSON Web Token (JWT).

No estado atual do projeto, o backend fornece base concreta para autenticação, persistência e organização dos principais fluxos do sistema. Entretanto, parte das interfaces ainda opera com dados mockados ou em memória, o que caracteriza uma etapa intermediária de integração entre os módulos.

De forma geral, a comunicação entre web, mobile e backend segue os seguintes princípios:

- uso de requisições HTTP/REST para troca de dados;
- autenticação por token JWT nas rotas protegidas;
- centralização das regras de negócio e da persistência no backend;
- consumo progressivo da API pelos módulos de interface.

Convenções de API

Padrão	Detalhe
Protocolo	HTTP/REST
Formato	JSON em requisições e respostas
Autenticação	JWT em rotas protegidas
Integração	Web e mobile usam o mesmo backend
Persistência	Centralizada no backend com PostgreSQL/Prisma

Exemplos de Endpoints

- POST /api/auth/register — cadastrar novo usuário
- POST /api/auth/login — autenticar usuário e obter token
- GET /api/users/me — obter perfil do usuário autenticado
- PUT /api/users/me — atualizar perfil do usuário autenticado
- GET /api/users — listar usuários (admin)
- GET /api/diary — listar entradas de diário
- POST /api/diary — criar nova entrada de diário
- PUT /api/diary/:id — atualizar entrada de diário
- DELETE /api/diary/:id — excluir entrada de diário
- GET /api/diary/admin/all — listar todos os diários (admin)

- GET /api/tips — listar dicas visíveis
- POST /api/tips — criar nova dica (admin)
- PUT /api/tips/:id — atualizar dica (admin)
- DELETE /api/tips/:id — excluir dica (admin)
- GET /api/faq — listar FAQs visíveis
- POST /api/faq — criar nova FAQ (admin)
- PUT /api/faq/:id — atualizar FAQ (admin)
- DELETE /api/faq/:id — excluir FAQ (admin)
- GET /health — verificação básica de disponibilidade da API
- GET /docs — documentação Swagger

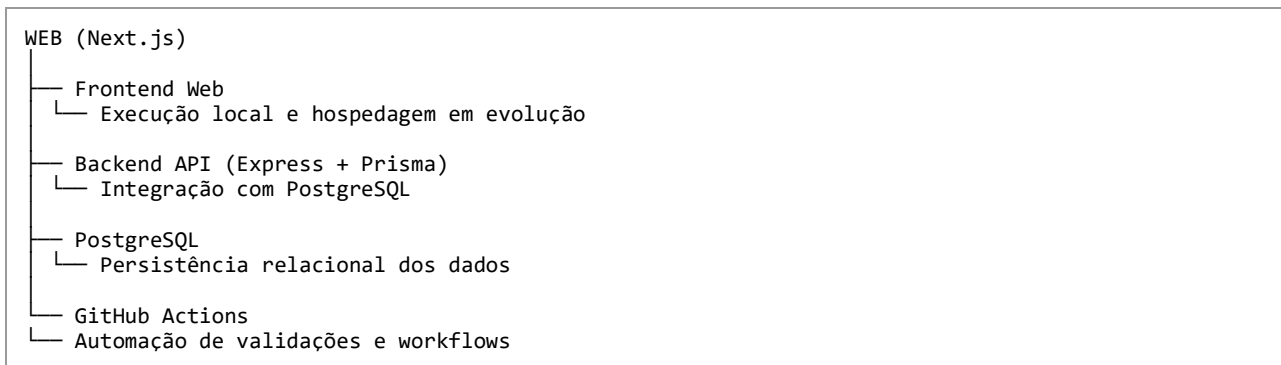
10 DEPLOY E INFRAESTRUTURA

10.1 Ambiente

No estado atual do projeto, o ecossistema DiabetesCare encontra-se em fase de desenvolvimento e integração progressiva entre os módulos principais. A execução do sistema ocorre predominantemente em ambiente local, utilizando dados mockados em parte das interfaces e integração gradual com o backend e o banco PostgreSQL. :

Ambiente	Finalidade	URL
Development	Desenvolvimento local com mocks	http://localhost:3000
Backend Local	Execução local da API	http://localhost:3333
Swagger	Documentação e testes da API	http://localhost:3333/docs

Infraestrutura Atual



10.2 Processo de Deploy

A execução local do projeto depende da instalação dos pré-requisitos necessários para cada módulo, incluindo Node.js, npm, PostgreSQL, Python e ambiente Expo para testes do aplicativo mobile.

No backend, o funcionamento depende da configuração de variáveis de ambiente, da conexão com o banco PostgreSQL, da execução das migrations e, quando necessário, da utilização do seed para dados iniciais. A documentação Swagger também pode ser acessada localmente para inspeção e teste dos endpoints implementados.

No painel web e no aplicativo mobile, parte dos fluxos ainda pode operar com dados mockados ou em memória, permitindo desenvolvimento e validação progressiva da integração com a API.

Como apoio ao fluxo técnico do projeto, o repositório já possui workflow de GitHub Actions voltado à automação de etapas relacionadas à validação e atualização do sistema. Ainda assim, outras definições de deploy e infraestrutura em produção não devem ser tratadas como componentes plenamente consolidados da versão atual.

Fluxo Geral de Execução

- instalação das dependências dos módulos;
- configuração das variáveis de ambiente;
- execução das migrations do Prisma;
- inicialização do backend;
- inicialização do frontend web;
- execução do aplicativo mobile via Expo;
- validação local dos fluxos principais do sistema.
- utilização de GitHub Actions para automação parcial de validações do projeto.

11 CONSIDERAÇÕES FINAIS

11.1 Pontos Fortes do Sistema

Experiência do Usuário

- Interface do paciente (mobile): design system coerente, paleta de cores clínica e intuitiva, feedback visual imediato em todas as ações, navegação clara com abas inferiores e hierarquia bem definida.
- Painel administrativo (web): visão consolidada e rica em dados, com gráficos informativos, tabelas com filtros avançados e código de cores clínico padronizado em toda a interface.
- Consistência visual: ambos os sistemas compartilham a mesma identidade visual, incluindo cores, tipografia e componentes, reduzindo a curva de aprendizado e reforçando a marca.

Arquitetura e Código

- TypeScript end-to-end: tipagem estática em todo o código, abrangendo web, mobile e API, eliminando uma classe significativa de erros em runtime e facilitando refatorações seguras.
- Separação de responsabilidades: estrutura de pastas clara e organizada, com camadas bem definidas, como componentes, hooks, utils, types e data.
- Design system robusto: tokens de cor, espaçamento, tipografia e bordas definidos centralmente, facilitando manutenção e evolução visual.
- Componentização: componentes pequenos, focados e reutilizáveis, seguindo o princípio da responsabilidade única.

Cobertura Funcional

- A plataforma cobre de maneira abrangente os principais aspectos do gerenciamento de diabetes: glicose, alimentação, medicação, hidratação, exercício, sono, metas e bem-estar emocional.
- O painel administrativo oferece ferramentas completas de moderação, monitoramento clínico e gestão de conteúdo, reduzindo a necessidade de ferramentas externas.

Escalabilidade Técnica

- Arquitetura stateless na API, permitindo escalabilidade horizontal sem dependência de sessões server-side.

- Cache Redis reduzindo a carga no banco de dados para dados frequentemente acessados.
- Deploy automatizado via CI/CD, minimizando riscos de erro humano e acelerando o ciclo de lançamento.

LINK DO PROJETO:

<https://github.com/FatecFranca/DSM-P5-G06-2026-01/blob/main/README.md>